

Asynchronous Result Pipelines in C#

C# における非同期 Result パイプライン: Task<Result> と LINQ ベースのモナディック合成による AI アプリケーション設計

Version: v0.2.0

DOI: 10.5281/zenodo.20458492

Author: Takuya Sogawa

ORCID: 0009-0009-7499-2595

License: CC-BY-4.0

Abstract

本稿は、C# における Task<Result<T>> と LINQ クエリ構文を用いた、実用的な非同期 Result パイプライン設計を定義する技術ノートである。このパターンは、Task<T> が表す非同期計算の文脈と、Result<T> が表す予測可能な失敗の文脈を合成する。AI アプリケーションでは、スキーマ不一致、モデル拒否、トークン長超過、安全でない出力、ツール拒否、明確化要求などは、必ずしも例外的なシステム障害ではない。それらは推論ドメインにおける通常の状態であり、値として表現されるべきである。

本稿では、Task<Result<T>> に対して Select と SelectMany を実装することで、C# のクエリ式を軽量なモナディック合成機構として利用できることを示す。これにより、予測可能な失敗を例外として扱わず、宣言的で Fail-Closed なショートサーキット型パイプラインを記述できる。

本稿の貢献は、Task<Result<T>> を単なる実装イディオムではなく、AI 時代の C# システムにおける契約指向の制御フロー・プリミティブとして位置づける点にある。すなわち、非同期、型付き、合成可能、AOT 互換、かつ統治された推論パイプラインに適した決定論的な失敗伝播モデルである。

1. Introduction

現代の C# ソフトウェアは、非同期処理を前提として構築される。ネットワーク呼び出し、ファイルアクセス、データベース問い合わせ、HTTP API、モデル Provider 呼び出しなどは、自然に Task<T> または関連する awaitable 型として表現される。

一方で、アプリケーションにおける失敗の多くは、必ずしも例外的ではない。入力不正である、AI モデルが拒否する、JSON スキーマに一致しない、Policy Gate が拒否する、下流ツールが明確化を要求する、といった結果は、通常の状態である。

例外は、予期しない障害、プログラム上の誤り、キャンセル、インフラ障害、回復不能な状態を表現するために重要である。しかし、通常の状態結果まで例外として扱うと、制御フローは読みづらくなり、失敗が型シグネチャから見えなくなる。

本稿は、予測可能な失敗を持つ非同期ワークフローを構造化するために、Task<Result<T>> を C# における実践的な抽象として提案する。このパターンは、不確実性、検証失敗、明確化、拒否、フォールバックが通常の状態として現れる AI アプリケーションに特に有効である。

2. 文脈を持つ値とモナディック合成

モナディックな設計では、値そのものだけでなく、その値が持つ文脈を合わせて扱う。本稿における文脈は次の三つである。

- Task<T>: 将来生成される非同期の値。
- Result<T>: 成功または構造化された失敗を持つ値。
- Task<Result<T>>: 成功値または予測可能な失敗を非同期に返す計算。

中心となる操作は Bind であり、C# の LINQ では SelectMany に対応する。Bind は、前の結果が成功した場合にのみ次の計算を実行する。前の計算が失敗していれば、パイプラインはショートサーキットし、その失敗をそのまま伝播する。

本稿は、C# に汎用的なモナドトランスフォーマーを導入するものではない。C# には高階型がないため、Task<Result<T>> という具体的な合成型に対して拡張メソッドを定義する実用的アプローチを採る。

3. Result 型の設計

最小限の実運用向け Result<T> は、成功時の値と失敗時の構造化エラーを持つべきである。デモ用には文字列エラーでもよいが、本番コードでは専用のエラー型が望ましい。

```
public readonly record struct Error(
    string Code,
    string Message,
    string? Detail = null);

public readonly struct Result<T>
{
    private readonly T? _value;

    public bool IsSuccess { get; }
    public bool IsFailure => !IsSuccess;
    public T Value => IsSuccess
        ? _value!
        : throw new InvalidOperationException("No success value.");
    public Error? Error { get; }

    private Result(T value)
    {
        IsSuccess = true;
        _value = value;
        Error = null;
    }

    private Result(Error error)
    {
        IsSuccess = false;
        _value = default;
        Error = error;
    }

    public static Result<T> Success(T value) => new(value);
    public static Result<T> Failure(Error error) => new(error);
}
```

この表現は、予測可能な失敗を値の世界に留める。プログラム上の誤りやインフラ障害に対して例外が発生することはありませんが、通常の検証・Policy・Provider 結果を表す主な手段は Result<T> となる。

4. Task<Result<T>> の LINQ 対応

C# のクエリ構文はパターンベースである。型が互換性のある Select と SelectMany を持っていれば、コンパイラはクエリ式をそれらのメソッド呼び出しへ変換できる。

Task<Result<T>> に対する Select と SelectMany は、非同期の Map と Bind を定義する。

```

public static class TaskResultExtensions
{
    public static async Task<Result<U>> Select<T, U>(
        this Task<Result<T>> source,
        Func<T, U> selector)
    {
        var result = await source.ConfigureAwait(false);
        if (result.IsFailure)
        {
            return Result<U>.Failure(result.Error!.Value);
        }

        return Result<U>.Success(selector(result.Value));
    }

    public static async Task<Result<U>> SelectMany<T, C, U>(
        this Task<Result<T>> source,
        Func<T, Task<Result<C>>> collectionSelector,
        Func<T, C, U> resultSelector)
    {
        var first = await source.ConfigureAwait(false);
        if (first.IsFailure)
        {
            return Result<U>.Failure(first.Error!.Value);
        }

        var second = await collectionSelector(first.Value)
            .ConfigureAwait(false);
        if (second.IsFailure)
        {
            return Result<U>.Failure(second.Error!.Value);
        }

        return Result<U>.Success(
            resultSelector(first.Value, second.Value));
    }
}

```

これにより、統治された非同期パイプラインを宣言的な列として記述できる。

```

Task<Result<ParsedData>> ExecuteAsync(string input)
{
    return
        from prompt in BuildPromptAsync(input)
        from output in CallModelAsync(prompt)
        from parsed in ParseOutputAsync(output)
        select parsed;
}

```

BuildPromptAsync、CallModelAsync、ParseOutputAsync のいずれかが失敗を返した場合、後続ステップは自動的にスキップされる。パイプラインは構造上 Fail-Closed となる。

5. AI アプリケーションにおける応用パターン

AI アプリケーションでは、典型的に次のような処理列が現れる。

1. プロンプトまたは構造化コンテキストを構築する。
2. モデル Provider を呼び出す。
3. 出力を検証する。
4. 応答をパースする。

5. Policy Gate を適用する。
6. 実行または結果返却を行う。

各ステップは予測可能な形で失敗し得る。

- 必須入力不足
- プロンプト構築失敗
- モデル拒否
- 不正な JSON
- スキーマ不一致
- Policy 拒否
- 安全でないツール呼び出し
- 信頼度不足
- 明確化要求

これらを `Result<T>` として表現することで、パイプラインはドメインの意味を保ったまま失敗を伝播できる。

```
Task<Result<Answer>> GovernedAnswerAsync(UserInput input)
{
    return
        from context in AssembleContextAsync(input)
        from prompt in GeneratePromptAsync(context)
        from modelText in CallModelAsync(prompt)
        from parsed in ParseJsonAsync(modelText)
        from decision in EvaluatePolicyAsync(parsed)
        from answer in RenderAnswerAsync(decision)
        select answer;
}
```

この記法では、正常系が上から下へ読みやすく表現され、失敗伝播は型に従って明示される。

6. Provider / Observer / Operator との関係

ILA の観点では、`Task<Result<T>>` パターンは Provider-Observer-Operator 抽象化規律と対応する。

Role	AI パイプラインでの例	失敗セマンティクス
Provider	model provider, VFS provider, context provider	unavailable, refused, not capable
Observer	schema validator, policy observer, risk observer	invalid, unsafe, ambiguous
Operator	parser, transformer, renderer, executor	transform failed, denied, incomplete

モナディックパイプラインは、これらの役割を消去しない。むしろ、それぞれの境界における結果契約を保ったまま、合成可能にする。

Provider は値を供給する。Observer は状態を評価する。Operator は状態を変換する。`Task<Result<T>>` は、この三役が共通の非同期失敗契約に参加するための形式である。

7. 例外と Result 値

本稿は例外を廃止することを提案しない。代わりに、次の二つを分離する。

Category	推奨される表現
予測可能なドメイン失敗	<code>Result<T></code>
検証失敗	<code>Result<T></code>
Policy 拒否	<code>Result<T></code>

Category	推奨される表現
明確化要求	<code>Result<T></code>
プログラム上の誤り	<code>exception</code>
インフラ障害	<code>exception</code> または変換された <code>failure</code>
キャンセル	<code>CancellationToken</code> / <code>cancellation exception</code>

例外と `Result` の境界はアーキテクチャ上の判断である。低レベルの HTTP タイムアウトは例外として発生し得るが、アプリケーションサービス層では `provider.timeout` のような `Result<T> failure` へ変換してもよい。

中心ルールは次の通りである。

予測可能なドメイン結果は値として伝播させる。予期しない障害は例外として扱ってよい。

8. Reactive Extensions との比較

Reactive Extensions (`IObservable<T>`) は、時間上に発生する値をモデル化する。イベントストリーム、UI イベント、センサーデータ、イベント駆動ワークフローに適している。

一方、`Task<Result<T>>` は、成功または失敗を持つ単一の非同期計算をモデル化する。これは、リクエスト・レスポンス型のワークフローや直列的な AI 推論パイプラインに適している。

Aspect	<code>IObservable<T></code>	<code>Task<Result<T>></code>
Cardinality	0 個以上の値	1 個の最終結果
Primary context	時間とストリーム伝播	非同期完了と失敗
Composition	stream operators	LINQ query / <code>SelectMany</code>
Error style	stream error channel	明示的な result value
Best fit	event streams	governed request pipelines

どちらも有用である。重要なのは、問題が持つ文脈に合った抽象を選ぶことである。

9. Rust / Haskell との比較

Rust は標準的に `Result<T, E>` を持ち、`?` 演算子によって早期リターンを簡潔に記述できる。本稿の C# パターンは、失敗をネストした条件分岐なしに伝播するという意味で、設計思想上これに近い。

Haskell の `Either` モナドと `do` 記法は、失敗し得る計算の逐次合成において類似の役割を果たす。C# の LINQ クエリ構文は、必要なクエリパターンを実装した型に対して、これに近い表層記法として利用できる。

違いは、C# には汎用的な高階型 `Monad` インターフェースがない点である。そのため、このパターンは具体的な `Select / SelectMany` 拡張メソッドとして実装される。

10. 現代 .NET における考慮事項

このパターンは、現代 .NET の設計制約と整合する。

- 通常の C# 型と拡張メソッドのみを使う。
- リフレクションや動的コード生成を必要としない。
- 周辺コードも AOT 互換であれば Native AOT と両立しやすい。
- `readonly struct` や `record struct` により不要な割り当てを抑制できる。
- ホットパスでは、プロファイル結果に基づき `ValueTask<Result<T>>` を検討できる。

`ValueTask` は、見た目が速そうだからという理由だけで使うべきではない。利用上の制約が増えるため、同期完了が頻繁に起きるなど、計測上の利益がある場合に選択すべきである。

11. Non-Claims and Limitations

本稿は以下を主張しない。

1. 例外が不要になったとは主張しない。
2. `Task<Result<T>>` がすべてのエラー処理の普遍的な代替であるとは主張しない。
3. C# に汎用的なモナドトランスフォーマーを提供するものではない。
4. すべての `Result<T>` 実装についてモナド則を証明するものではない。
5. LINQ クエリ構文が常に明示的なメソッド呼び出しより優れているとは主張しない。
6. すべての AI failure が予測可能なドメイン失敗であるとは主張しない。

本稿の主張はより限定的である。すなわち、予測可能な検証失敗、Policy 失敗、パース失敗、Provider 失敗を持つ非同期 AI ワークフローにおいて、`Task<Result<T>>` は明確で型指向の合成モデルを提供する。

12. Conclusion

`Task<Result<T>>` は、非同期実行と決定論的な失敗伝播を合成する必要がある C# AI アプリケーションにとって自然な抽象である。予測可能な失敗を値へ変換し、エラーセマンティクスを型シグネチャに表出させ、`Select` と `SelectMany` による LINQ ベースの宣言的パイプラインを可能にする。

AI 時代において、不確実性は常に例外ではない。しばしばそれ自体がドメインである。よく設計された `Result` パイプラインは、その不確実性を統治し、合成し、検証し、安全にショートサーキットさせる。

Interface-Led Architecture の観点では、このパターンは非同期推論を `Contract` を保存するパイプラインへ変換する。Provider は値を供給し、Observer は状態を検証し、Operator は出力を変換し、失敗はすべての境界で明示されたまま維持される。

References

1. Microsoft. “Task asynchronous programming model.” Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/task-asynchronous-programming-model>.
2. Microsoft. “Projection operations in LINQ.” Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/dotnet/csharp/linq/standard-query-operators/projection-operations>.
3. Microsoft. “C# language specification: Query expressions and query expression translation.” Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/expressions>.
4. Microsoft. “Best practices for exceptions.” Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/dotnet/standard/exceptions/best-practices-for-exceptions>.
5. Microsoft. “Native AOT deployment overview.” Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/>.
6. Microsoft. “ValueTask Struct.” Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/dotnet/api/system.threading.tasks.valuetask-1>.
7. The Rust Project Developers. “Recoverable Errors with Result.” The Rust Programming Language. Available at: <https://doc.rust-lang.org/book/ch09-02-recoverable-errors-with-result.html>.
8. Wadler, Philip. “Monads for Functional Programming.” In *Advanced Functional Programming*, Lecture Notes in Computer Science, vol. 925, Springer, 1995, pp. 24-52. DOI: 10.1007/3-540-59451-5_2.
9. ReactiveX. “ReactiveX: An API for asynchronous programming with observable streams.” Available at: <http://reactivex.io/>.
10. Sogawa, Takuya. “Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model.” Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
11. Sogawa, Takuya. “Provider-Observer-Operator: A Role-Based Abstraction Discipline for Interface-Led Architecture.” Zenodo, 2026. DOI: 10.5281/zenodo.20322690.
12. Sogawa, Takuya. “Prompt-State Machines: Applying Interface-Led Architecture to Structure LLM Reasoning.” Zenodo, 2026. DOI: 10.5281/zenodo.20323512.